Quantum Modelling of Time Series: Expressivity vs. Trainability

Jacob L. Cybulski

jacob.cybulski@deakin.edu.au

Deakin University

Sebastian Zajac

SGH Warsaw School of Economics

Research Article

Keywords: Quantum time series analysis, Quantum model architecture, Quantum model optimization, Quantum machine learning

Posted Date: August 20th, 2025

DOI: https://doi.org/10.21203/rs.3.rs-7253212/v1

Additional Declarations: No competing interests reported.

Quantum Modelling of Time Series: Expressivity vs. Trainability

Jacob L. Cybulski[1*] and Sebastian Zajac[2†]

[1*]School of IT, Deakin University, Melbourne, Vic, Australia.
[2]SGH Warsaw School of Economics, Warsaw, Poland.

*Corresponding author(s). E-mail(s): jacob.cybulski@deakin.edu.au;
Contributing authors: szajac2@sgh.waw.pl;
†These authors contributed equally to this work.

Abstract

As time series feature unstructured data and unique processing requirements, they do not easily fit conventional approaches to quantum modeling. This paper therefore presents various quantum model architectures suitable for encoding and analysis of time series data. In particular, it investigates their architectural design aspects, such as methods of encoding and reuploading of temporal data, parameterizing quantum circuits, controlling the circuit qubit resources and entanglement, and dealing with issues of state evolution in the model Hilbert space, as well as navigability of the classical parameter space for an optimizer. Each approach enhances or impedes model expressivity, i.e. its ability to effectively represent time series data in quantum space, as well as its trainability, i.e. its capacity to learn and generalize for predictive accuracy and efficiency in the process of model optimization. The paper evaluates different quantum circuit configurations for their impact on the expressivity and trainability of a quantum time series analysis model, and analyzes how these two design qualities influence model performance in training and testing.

Keywords: Quantum time series analysis, Quantum model architecture, Quantum model optimization, Quantum machine learning.

1 Introduction

Time series (TS) analysis and forecasting is a well-established branch of mathematics, statistics, economics and machine learning (ML). Over the years, ML has expanded

its scope by borrowing concepts from other disciplines, most recently from quantum mechanics, resulting in the ML subfield of quantum machine learning (QML). Classical ML successfully addressed the entire spectrum of time series processing methods [1], especially with the introduction of deep learning (DL) methods and technologies [2]. At the same time, QML did not achieve many breakthroughs in quantum time series analysis (QTSA). Existing quantum applications include those in signal processing [3], time series classification [4], forecasting [5], and anomaly detection [6].

All such TS applications propose quantum models to deal with data sequences, data that are distinct from tabular data, and which require special approaches to their representation and processing. The design of such models needs some unique methods of data encoding for the preparation of the quantum state, appropriate circuit architectures for the processing of data features in multidimensional quantum space, and suitable measurements to extract classical information from the circuit state. There are also many design factors that impact the performance of the TS models training and the consequent effectiveness of their application to the intended analytical tasks. In this article, we considered only the models' fundamental predictive abilities, such as in data fitting or forecasting, but these abilities are applicable to many other analytical tasks. The scope of the model features will also be limited to those relevant to quantum neural network design. In particular, we investigated the architectures that allow circuit under- and overparametrisation [7, 8], we experimented with encoding techniques that reject or embrace data re-uploading [9], and we proposed and tested the circuits ability to overload the qubit state for data encoding.

2 QTSA models

The work reported in this article was conducted in the context of gate-based quantum computing. Within this framework, circuit gates are commonly grouped into functionally distinct blocks, such as an input block encoding classical data to set the circuit's initial quantum state, a trainable ansatz transforming this state, and an output block preparing and measuring the final circuit state. Circuit blocks, and ansatze in particular, could be further split into similarly structured layers of gates, hence allowing stepwise state transformations. However, there are many other quantum model architectures that feature unique arrangements of gates and their structures.

Variational quantum algorithms (VQA) are commonly used to optimize quantum circuits [10]. VQAs optimize the model parameters using classical optimization algorithms, by gradually changing the parameter values to satisfy the criteria of some objective function (in terms of cumulative error expressed as loss or cost). As the process of VQA processing involves both quantum computation and classical optimization, the approach is referred to as a hybrid quantum-classical method.

QTSA methods adopt and adapt existing modeling approaches and model optimization processes to efficiently represent and manipulate data sequences. The quantum circuit structures and their processing depend largely on the task at hand.

Table 1 QTSA approaches

In:Out	Eq	Constraints (q = qubit#)	Model	Task
1:1	3	$f(x) \rightarrow y, q = 1$	serial	TS curve-fitting (single qubit)
1:1	7 8	$f(x) \rightarrow y, 1 < q$	parallel xparallel	TS curve-fitting (multiple qubits)
n:1	1	$f(w) \rightarrow y, w = (x_1, ..., x_n), y \in \{0, 1\}$	qnn	TS classification
	1	$f(w) \rightarrow y, w = (x_1, ..., x_n), y \in R, n \leq q$		TS estimation
n:k	13	$f(w) \rightarrow h, w = (x_1, ..., x_n), h = (h_1, ..., h_k)$ $n \leq q, k \leq q$	xqnn	TS forecasting (short window)
n:k	16	$f(w) \rightarrow h, w = (x_1, ..., x_n), h = (h_1, ..., h_k)$ $q \leq n, k \leq q$	ovload	TS forecasting (long window)
n:n		$f(w) \rightarrow v, w = (x_1, ..., x_n), v = (v_1, ..., v_n)$ $n \leq q$	qae/qgan	TS mapping

Note: *TS - time series*
Eq - forward reference to an equation

The tasks may include (see Table 1):

- *curve-fitting*: able to map a point in time to an approximation of data value, such models are of use for prediction or explanation of data properties;
- *classification / estimation*: able to map a sliding window of TS values to either a class or a single value not present in the sequence, such models are of use for decision making;
- *forecasting of future TS data points*: able to map a sliding window of TS values to a horizon of future values (note that windows could be longer than the number of circuit qubits), such models are suitable to support forecasting tasks; and
- *mapping between TS sequences*: able to map a sliding window of TS values to a window of transformed TS values, such models can be used to reduce signal noise or improve its quality.

In this article, we focus on the predictive tasks, i.e. curve-fitting and forecasting.

Each QTSA task can be performed by a quantum model of a specific form and function. In our research, we rely on quantum neural networks (QNNs) [5], which can be used for a wide range of analytics tasks, such as classification or estimation. QNNs exhibit a generic and regular circuit structure; however, they can be extended in a variety of different ways beyond the standard input-ansatz-output sequence of blocks.

For example, the basic QNN architecture can be enhanced with auxiliary qubits to gain parameters and increase model ***expressivity*** [11, 12]. The addition of qubits to the model also increases its entanglement, which is crucial to the expressivity of quantum models; however, the excess of entanglement causes *nonlocal interactions* between distant parameters, which reduces its ***trainability***, especially in gradient-based optimization [13]. Furthermore, the high dimensionality of the model parameter space leads to *flattening the gradient space* of the model geometry, leading to the development of barren plateaus and thus impeding optimization of the model parameters

[14, 15]. At the same time, the addition of model parameters beyond a critical threshold can lead to circuit *overparameterisation*, which can benefit quantum models by increasing their learning capacity and, consequently, their trainability [7]. The circuit can be structured to repeatedly reupload and intersperse input data with layers of trainable gates [9], known to improve the *representation of data features* in a Hilbert space of high dimensions and the subsequent improvement of model performance [16]. In addition, some data encoding techniques, relying on mid-circuit measurements or qubit reinitialization, can decrease demand for qubit utilization [17], leading to a narrow circuit and consequent reduction in its entanglement. In general, in this paper, we investigate how QNN extensions could balance expressivity and trainability, and add value to two separate groups of predictive quantum time series models.

The first group are ***curve-fitting*** models. Such models are trained to fit a function to a sample of data points with the objective of predicting data values at specific points in time. Three types of curve-fitting models were considered, all three adopting data reuploading, but each using a different method. The methods included: sequential data reuploading within a single qubit - *serial* (Eq. 3 and Figure 4), reuploading across multiple qubits - *parallel* (Eq. 7 and Figure 5), and sequential multi-qubit reuploading - *xparallel* (Eq. 8 and Figure 6).

The second group are ***forecasting*** models able to predict future data points from their preceding temporal context, which are captured in a fixed-size window sliding over the time series. Quantum forecasting models set their circuit's state by encoding a sliding window (SW) of data, and subsequently, after its processing, measuring the final state and decoding a sequences of future data, which form a multivalue TS horizon.

Two QNN variants were considered for forecasting. The first variant was the *xqnn* model capable of encoding data windows across qubits dedicated to input encoding, while extending the width of the QNN circuit to allow its overparameterisation (Eq. 13 and Figure 7). The second variant was the *ovload* model, which overcomes the QNN limitation, which prevents the encoded input (TS window) from exceeding the number of available qubits (circuit width). The model accomplishes this by splitting long TS windows into fragments that can then be encoded between layers of trainable gates (Eq. 16 and Figure 8). The 16 model thus overloads the representational function of quantum qubits, reusing the qubits and thus reducing the demand on the qubit resources. Overloading takes advantage of the high redundancy between adjacent TS window values. This approach also allows encoding windows from multivariate series (not covered here).

While there exist many ways of embedding data in a quantum circuit, e.g. basis, amplitude, QRAM and angle encoding [18], it was necessary to select only those methods that have the potential of representing time series data in forecasting models, i.e. they could encode windows of multiple floating point numbers. Angle encoding, which invokes the rotation of the qubit states by the specified angular value [10], was deemed the best suited for this purpose. While amplitude encoding is also suitable for the representation of time series windows, it varies the circuit structure, impairing its efficient execution on some quantum devices. Although curve-fitting models require

encoding a single floating point number in their circuits, for consistency reasons, angle encoding was used for both forecasting and curve-fitting models.

Although measurement strategy is an important and vigorously studied aspect of model training, for the sake of brevity, it was excluded from the scope of this article. Hence, all adopted QNN variants used the identical approach, measuring all their qubits, thus monitoring the circuit's global properties. For the same reason, the impact of noise on quantum computation was left out of this study.

It is important to note that a classical neural network model (*cnn*) was also included for comparison and a performance benchmark for the included quantum models. Its architecture places it among the forecasting models which rely on the sliding window version of the data.

All models were implemented in Qiskit 1.3.2, with Machine Learning 0.8.2 and PyTorch 2.6.0. The models were trained on Ubuntu Linux 22.05.5 LTS, with 64Gb RAM and 13th-gen Intel Core i9 x 32. At that time, the Qiskit machine learning module did not support GPU effectively, and hence all calculations were conducted using Qiskit Estimator primitives and its state-vector simulator, with CPU support only.

3 Formalization of QTSA architectures

Each QTSA model extends a generic quantum neural network (QNN), which is expressed as a parameterized quantum circuit:

$$U(\vec{x}, \vec{\theta}) = U_{\text{ansatz}}(\vec{\theta}) \cdot \mathcal{U}_{\text{FM}}(\vec{x}), \tag{1}$$

where $\vec{x}$ denotes the classical input data, $\vec{\theta}$ are all trainable parameters, $\mathcal{U}_{\text{FM}}$ is the unitary data encoding circuit (the feature map [19]) and U_{ansatz} represents trainable quantum transformations.

A prediction is extracted by measuring the expectation value of an observable M:

$$y = \langle 0|^{\otimes q} U^{\dagger} M U |0\rangle^{\otimes q}, \tag{2}$$

where q denotes the number of qubits, and unless specified otherwise, $M = \sum_{j=1}^{q} Z_j$.

3.1 Curve-Fitting Models

The QTSA models fitted with curvatures aim to approximate a continuous function $f : \mathbb{R} \to \mathbb{R}$ to predict individual values of the time series step by step. These models are characterized by reuploading the same scalar input $x \in \mathbb{R}$ multiple times throughout the quantum circuit to capture non-linear temporal dependencies. As discussed above, we consider three architectural variants:

(a) Serial Reuploading (serial).

In the single-qubit case ($q = 1$), the circuit alternates between trainable unitary blocks and repeated data-encoding gates. Each data-encoding gate is a rotation about the

X-axis, while each trainable gate is a general single-qubit unitary parameterized by three angles.

Given a scalar input x and number of layers n, we define the parameter set as $\vec{\theta} = \{\boldsymbol{\theta}_1, \ldots, \boldsymbol{\theta}_{n+1}\}$, where each $\boldsymbol{\theta}_k = (\theta_k^{(1)}, \theta_k^{(2)}, \theta_k^{(3)})$.

The resulting quantum state is given by:

$$\left|\psi(x, \vec{\theta})\right\rangle = U(\boldsymbol{\theta}_{n+1}) \cdot \left(\prod_{j=1}^{n} \left[R_X(x) \cdot U(\boldsymbol{\theta}_j) \right] \right) \cdot |0\rangle \,, \tag{3}$$

where $R_X(x) = \exp(-ixX/2)$ is the data encoding operation.

(b) Parallel Reuploading (parallel).

In the multi-qubits case (q ¿ 1), this architecture involves reuploading data to a register of q qubits and encoding the same scalar input x identically in each:

$$R_X^q(x) = \bigotimes_{j=1}^{q} R_X^{(j)}(x). \tag{4}$$

Variational layers before and after the encoding consist of entanglement blocks:

$$U^q(\boldsymbol{\theta}_j) = \bigotimes_{i=1}^{q} U^{(i)}(\boldsymbol{\theta}_j^{(i)}), \tag{5}$$

and

$$U_{\text{ent}}(\boldsymbol{\theta}_j) = \prod_{i=1}^{q} \text{CNOT}_{i,\,(i \bmod q)+1} \cdot U^q(\boldsymbol{\theta}_j), \tag{6}$$

where each $U^{(i)}$ represents a trainable single-qubit unitary operator, and each $\boldsymbol{\theta}_k$ consists of a block of q parameter vectors, resulting in a total of $3 \times q$ parameters for the k-th layer.

The full quantum state is prepared as:

$$\left|\psi(x, \vec{\theta})\right\rangle = \left(U^{\text{q}}(\boldsymbol{\theta}_{n+1}) \prod_{j=m+2}^{n} \left[U_{\text{ent}}(\boldsymbol{\theta}_j) \right] \right) R_X^{\text{q}}(x) \left(U^{\text{q}}(\boldsymbol{\theta}_{m+1}) \prod_{k=1}^{m} \left[U_{\text{ent}}(\boldsymbol{\theta}_k) \right] \right) |0\rangle^{\otimes q} \,, \tag{7}$$

where $\vec{\theta}$ is the full parameter set and n, m are the numbers of layers after and before encoding, respectively.

(c) Extended Parallel Reuploading (xparallel).

This variant interleaves reuploading layers with entangling blocks multiple times, improving capacity to model long-term dependencies. The prepared state is:

$$\left|\psi(x,\vec{\theta})\right\rangle = \left(U^q(\boldsymbol{\theta}_{m+1}) \prod_{k=L+1}^{m} U_{\text{ent}}(\boldsymbol{\theta}_k)\right) \cdot \left(\prod_{l=1}^{L} [R_X^q(x) \cdot U_{\text{ent}}(\boldsymbol{\theta}_l)]\right) |0\rangle^{\otimes q}, \tag{8}$$

where L is the number of reuploading layers.

3.2 Forecasting Models

Forecasting QTSA models aim to predict a sequence of future time-series values from a given input window. These models approximate mappings $f : \mathbb{R}^n \to \mathbb{R}^k$, where n is the window size and k is the prediction horizon.

(a) Extended QNN (xqnn)

The architecture can be interpreted as an extension of the formulation given in Eq. (1), where the feature map $U_{FM}(\hat{\mathbf{x}})$ encodes the input vector $\hat{\mathbf{x}} = (x_1, \ldots, x_n)$, consisting of n features, into a quantum state.

The employed **ZZ feature map** spans only n qubits (assuming $n \leq q$). We follow the approach introduced in [20], where the feature map on n qubits is defined as:

$$\mathcal{U}_{FM}(\mathbf{x}) = U_{\Phi(\mathbf{x})} \cdot H^{\otimes n}, \tag{9}$$

where H denotes the Hadamard gate and

$$U_{\Phi(\mathbf{x})} = \exp\left(i \sum_{S \subseteq [n]} \phi_S(\mathbf{x}) \prod_{i \in S} Z_i\right). \tag{10}$$

Only subsets S with cardinality $|S| \leq 2$ are considered. In particular, $\phi_{\{i\}}(\hat{\mathbf{x}}) = x_i$ and $\phi_{\{i,j\}}(\hat{\mathbf{x}}) = (\pi - x_i)(\pi - x_j)$. Each term $\exp\left(i\phi_{\{i,j\}}(\hat{\mathbf{x}}) Z_i Z_j\right)$ can be implemented using a combination of CNOT and R_Z gates (see Fig. 1(c) in [20]).

Alternatively, the feature map can be expressed using a layered implementation:

$$U_{\Phi(\mathbf{x})} = \left(\prod_{j=1}^{L_{FM}} \left(\prod_{k=1}^{n} R_Y(x_k)\right) \left(\prod_{\substack{k=1 \\ k \text{ odd}}}^{n-1} R_{ZZ}(x_k x_{k+1})\right) \left(\prod_{\substack{k=2 \\ k \text{ even}}}^{n-1} R_{ZZ}(x_k x_{k+1})\right)\right) \cdot \left(\prod_{k=1}^{n} R_Y(x_k)\right). \tag{11}$$

The variational ansatz $U_{AN}(\boldsymbol{\theta})$ is a parameterized quantum circuit that acts on all q qubits. It consists of L_{AN} layers of trainable unitary blocks $U_{\text{ent}}(\boldsymbol{\theta}_j)$ (see Eq. 6), where each $\boldsymbol{\theta}_j$ denotes the parameter set for the j-th layer.

The total ansatz unitary is then:

$$U_{AN}(\boldsymbol{\theta}) = \prod_{k=1}^{L_{AN}} U_{\text{ent}}(\boldsymbol{\theta}_k). \tag{12}$$

The resulting quantum state is prepared by applying the feature map followed by the variational ansatz to the initial state:

$$|\psi(\mathbf{x}, \boldsymbol{\theta})\rangle = U_{AN}(\boldsymbol{\theta}) \cdot \mathcal{U}_{FM}(\mathbf{x}) \, |0\rangle^{\otimes q} . \tag{13}$$

(b) Overloaded QNN (ovload)

When the number of input features exceeds the number of qubits, i.e., $n > q$, the input vector $\hat{\mathbf{x}} = (x_1, \ldots, x_n)$ is cyclically assigned to the available qubits. Specifically, features are grouped into $\lceil \frac{n}{q} \rceil$ segments, and within each segment, the values are mapped to the corresponding qubits via repeated applications of the rotation gate R_X. The feature encoding is then given as:

$$\mathcal{R}^q X(\hat{\mathbf{x}}) = \prod k = 0^{\lceil n/q \rceil - 1} \left(\bigotimes_{j=1}^{q} R_X^{(j)} \left(x_{kq+j} \right) \right), \tag{14}$$

where we define $x_m = 0$ for any index $m > n$ to pad the input if n is not divisible by q.

Each entanglement layer is formed by combining two unitary blocks:

$$W_{\text{ent}}^{l}(\hat{\boldsymbol{\theta}}) = U^q(\boldsymbol{\theta}_{l+1}) \cdot U_{\text{ent}}(\boldsymbol{\theta}_l), \tag{15}$$

where $\hat{\boldsymbol{\theta}}$ includes all sets of parameters.

The final quantum state is constructed by repeatedly applying input encoding and entangling blocks.

$$\left| \psi(\hat{\mathbf{x}}, \hat{\boldsymbol{\theta}}) \right\rangle = W_{\text{ent}}^{L+1}(\hat{\boldsymbol{\theta}}) \cdot \left(\prod_{k=1}^{L} \left[\mathcal{R}_X^q(\hat{\mathbf{x}}) \cdot W_{\text{ent}}^{k}(\hat{\boldsymbol{\theta}}) \right] \right) |0\rangle^{\otimes q} . \tag{16}$$

This "overloaded" structure increases expressivity by coupling each qubit directly to an input dimension and adding more variational flexibility per layer.

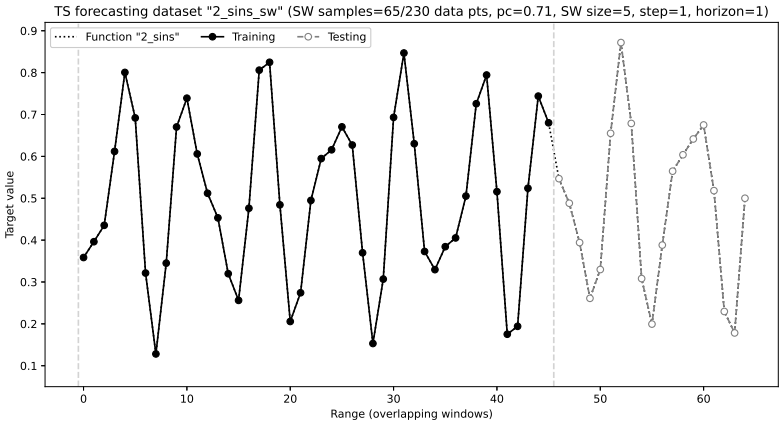

Fig. 1 Sample synthetic data set
$f(x) = \frac{sin(5.0*x)+0.5*sin(8.0*x)}{4} + 0.5$

4 TS data and their preparation

For training of all curve-fitting models, we used a simple synthetic data set *2_sins*, which combined trigonometric functions to generate an oscillation pattern of 70 data points, normalized to the value range of (0–1). The data series was then divided into 50 data points for training and 20 data points for testing. Data for forecasting models were prepared differently, as the 70 *2_sins* data points were first converted to sliding windows of 5 data points, windows were offset by 1 step, with the horizon of 1, and all incomplete windows were rejected. The resulting series of 65 windows was subsequently split into 46 windows for training and 19 windows for testing (see Figure 1). In both cases, data were divided into training and testing partitions in an approximate ratio of 71/29. Unlike tabular data, time series data can present significant challenges for model training, due to the close proximity of neighboring values in a time series. In curve-fitting models, the optimizer will have to deal with groups of very similar values provided on input and expected on output. In forecasting models, the optimizer will struggle with features of the TS window that are similar in amplitude and lack independence. It is hence crucial that the quantum model provides an architecture that maps these features into a high-dimensional Hilbert space in a way that helps the optimization tasks.

Several commonly performed data preprocessing steps were not required for the selected TS dataset and the development of QTSA models, due to a small size of the synthetic data adopted and the reliance on noise-free quantum simulators.

These included dimensionality reduction and time series compression, imputation of missing values, feature engineering and temporal structure discovery, noise removal, dealing with outliers and anomalies, strict adherence to data normality and homoskedasticity, and data shuffling (not applicable to time series).

5 QTSA models performance

The effectiveness of each model type was investigated by changing its architectural meta-parameters, such as the number of qubits and layers, and their roles. This resulted in distinct model architectures. Each architecture was then instantiated 10 times, with each instance randomly initialized, and then trained to achieve a minimum cost (in our case MSE). At each optimization step, all the parameters of the model instance were saved for later scoring in training and testing data partitions. The scoring metrics included MAE, MSE, and R^2. The performance of a specific model architecture (within the model type) was then represented by the instance that resulted in a median optimum test score (maximum R^2).

All Qiskit QML models were implemented with *EstimatorQNN* primitives, designed to compute the expectation values of quantum observables. The models were trained utilizing the *NeuralNetworkRegressor* performing regression tasks for Qiskit QNNs. The regressor relied on the limited-memory version of the Broyden-Fletcher-Goldfarb-Shanno (L_BFGS_B) algorithm, designed as an iterative method for solving unconstrained, large-scale nonlinear optimization problems. The L_BFGS_B optimization process of every quantum model was guided by the *L2Loss* function (MSE cost), which was set to complete in 60 training epochs, which was sufficient for all models to converge (note that for each iteration, internally the L_BFGS_B algorithms can execute up to 1000 function applications).

After training, performance measurement involved scoring all models with several training and testing statistics and then comparing the models within and across curve-fitting and forecasting groups.

It is important to note that, in general, models of both groups have unique objectives in this process. Curve-fitting models aim to minimize the error at each specific point in time. The residuals are directly linked to individual data points. Forecasting models, on the other hand, aim to predict future values based on a temporal context of sliding windows. The residuals are measured on the forecasted values, which is drastically different from the historical points the curve-fitting models operate on. In both cases, the notions of "data point" and "error" between the model groups are quite different. It is worth noting that these two notions are identical for all models in their respective groups, and hence the MAE and MSE statistics were useful for a later comparison of performance in training and testing within their group. However, due to inconsistencies in the fundamental measurement concepts, MAE and MSE were not appropriate to compare the models and their performance between the curve fitting and forecasting groups, especially because the forecasting models relied on the overlapping window data, thus covering a significantly larger set of (repeated) data points (230) than those of the curve fitting models (70).

Hence, it was decided to compare performance between two groups of models based on some relative measure between prediction and ground truth, such as MAPE, sMAPE, NRMSE or R^2 statistic (coefficient of determination). R^2 was selected for its simplicity and widespread use. R^2 is a proportion of variance in the dependent variable that can be explained by independent variables. So, the R^2 statistic uses a very simple baseline - the mean of the dependent variable, it is unit-free and scale independent.

Table 2 Results - comparison of QTSA models' performance

Model	Type	Qubits	Params	Training R^2	Testing R^2	Specs
serial	curve-fitting	1	66	0.9966	0.9192	q1 l21
serial	curve-fitting	1	84	0.9977	0.9107	q1 l27
parallel	curve-fitting	5	120	0.8460	0.6957	q5 bl3 al3
parallel	curve-fitting	5	90	0.8145	0.7137	q5 bl1 al3
xparallel	curve-fitting	3	81	0.9980	0.9823	q3 bl7 al1
xqnn	forecasting	7	105	0.9858	0.8478	q7 in5 fm1 anz4
xqnn	forecasting	7	84	0.9603	0.8796	q7 in5 fm1 anz3
ovload	forecasting	4	167	0.8890	0.8735	q4 xl3 il2
cnn	forecasting	0	6181	1.0000	0.9797	hl050 080 020

This makes it a better candidate for cross-group comparison than absolute error metrics such as MSE or MAE, although R^2 measures the goodness of fit using variance rather than error.

6 Summary of experimental results

Table 2 summarizes and compares the experimental results within and between two groups of models using R^2 statistic. For each model architecture, the table reports the median R^2 score (highlighted) achieved for all model instances in training and testing, which can be viewed in one or two rows of the table.

The performance of individual model architectures is reported as that achieved by its instance of median performance. This approach has some advantages, as some models are highly sensitive to their initialization and perform extremely poor, thus generating outliers (e.g., small *cnn* models, see Figure 9) or skewing Gaussian distribution of instance results. In such circumstances, an effective calculation of their mean is impossible. The additional advantage is that the median performance is attached to a specific model instance, so that it could be used to visualize its data fit (for even number of results, the first of the middle pair is used).

Among curve-fitting models (*serial*, *parallel* and *xparallel*), reuploading data serially leads to models fitting data better (training R^2=0.9977 and testing R^2=0.9192) than those reuploading data in a purely parallel fashion (training R^2=0.8460 and testing R^2=0.7137). However, the best model in terms of training and test performance was *xparallel*, which incorporates serial and parallel re-uploading methods (training R^2=0.9980, testing R^2=0.9823). Although the *xparallel* model relies on a smaller number of training parameters than the *parallel* model (81 vs 90 parameters), its testing performance was close or slightly better than the performance of much larger classical neural networks (for example, at 6181 parameters, testing R^2=0.9797). Figure 2 illustrates a nearly perfect fit of the data to the median performing model instance from the best curve-fitting *x-parallel* model architecture in the training data partition and an excellent fit in the test partition.

Forecasting models are represented by two variants of QNNs (*xqnn* and *ovload*). Training of *xqnn* models, which allow circuits to be overparameterized, improves with the width of the circuit, as their training performance increases with added qubits (up to training R^2=0.9858). However, beyond a certain point, adding more circuit layers and parameters reduces both training and testing performance (best testing

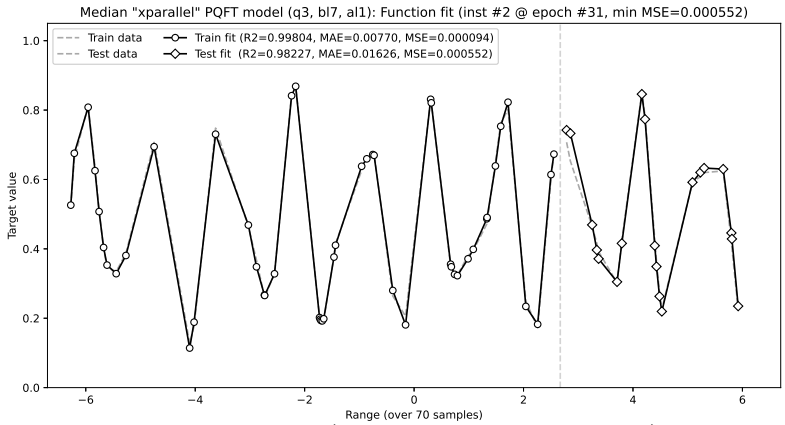

Fig. 2 Data fit of the curve-fitting model (*xparallel* of median performance). Note: training partition - left, test partition - right

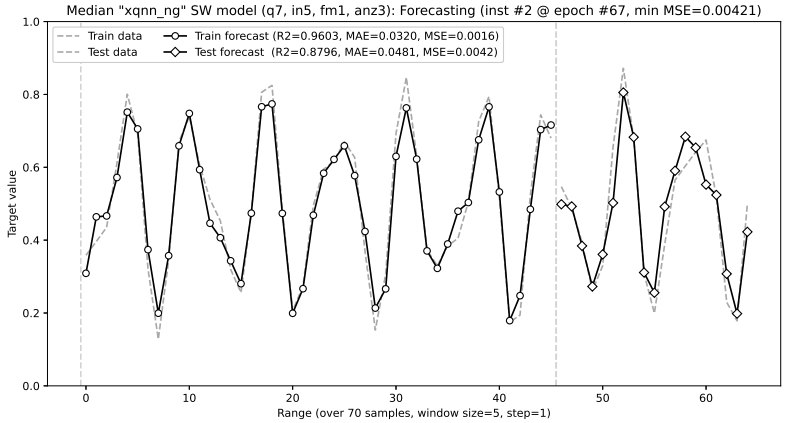

Fig. 3 Data fit of the forecasting model (*xqnn* of median performance). Note: training partition - left, test partition - right

R^2=0.8796 at 84 parameters and 3 layers). The *ovload* models, which are capable of overloading the qubit parameters, do not have the same level of learning capacity as *xqnn* models (training R^2=0.8890 only). However, they achieve the equivalent level of test performance (testing R^2=0.8735), and at a reduced number of qubits (only 4 vs 7 qubits), however, at the expense of doubling the number of model parameters (167 vs. 84). Figure 3 illustrates a good fit of the data to the median performing model instance of the best *xqnn* model architecture in the training and testing data partitions.

The next section provides a more detailed analysis of the experimental results, necessary to fully understand the architectural effects on models performance.

7 Analysis and discussion

Critical to this analysis is understanding the relationships between different aspects of model design. These aspects include:

- *Hilbert space* (defined by the number of qubits) is the quantum realm where the model state evolves (defined by the circuit) in response to unitary operations (defined by circuit gates);
- *data encoding* brings in classical data into the Hilbert space as unique and correlated quantum states during the model execution;
- *layers of unitary operations* (defined by circuit gates) determine the evolution of the quantum model's initial state into its final state;
- *trainable parameter space* is a classical multi-dimensional space of trainable parameters (defined by circuit gates), which the optimizer navigates;
- *entanglements* (defined by CNOT gates) create and correlate non-separable qubit states, which during the model execution and its state evolution alter the parameter space geometry, and consequently the landscape of the cost function used by the optimizer;
- *measurement* of individual qubits projects their final quantum states onto classical outcomes, the probabilities of which reflect their correlation.

As discussed earlier (see Section 1), more qubits, higher circuit entanglement, data reuploading, and overparameterization can contribute to model expressivity [7, 9, 11, 12, 16]. On the other hand, poor data encoding, more layers and parameters, excessive entanglement, and nonlocal interactions can reduce model trainability [13–15]. At the same time, relying on mid-circuit measurements and qubit reuse can improve the model trainability [17].

Therefore, depending on the adopted model architecture, each of these design aspects may have a beneficial or detrimental effect on the expressivity and trainability of the model. The following analysis of experimental results will explicitly address these aspects in the discussion.

All models presented in this section are illustrated with a sample quantum circuit and described in a table of the model's architectural variations. Each of the variants is annotated with a description of its structure in terms of the number of qubits and layers that play different roles in the circuit, as described within the section devoted to its analysis. All circuits measured expectation values on all qubits individually. Therefore, although the measurement strategy may have a significant impact on model trainability (local vs. global, dimensionality, sparsity, and precision), in this study its impact has not been taken into account.

7.1 Experiments with curve-fitting models

7.1.1 Serial models

The curve-fitting *serial* models (see Figure 4 and Table 3) were unique, since they used only a single qubit (q1), with several layers (l) of data reuploading and trainable blocks.

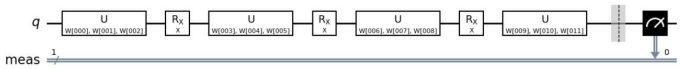

Fig. 4 Sample serial model (Arch specs: q1 l3)

Table 3 Performance of a "serial" (2-sins) model type

		Training		Testing		Arch
Qubits	Params	R^2	MSE	R^2	MSE	Specs
1	12	-0.0495	0.0503	-0.1320	0.0353	q1 l3
1	30	0.7768	0.0107	0.5166	0.0151	q1 l9
1	48	0.9616	0.0018	0.7514	0.0077	q1 l15
1	66	0.9966	0.0002	0.9192	0.0025	q1 l21
1	84	0.9977	0.0001	0.9107	0.0028	q1 l27

Because of repeated data reuploading interleaved with parametrized state rotations, the circuit implements a highly dimensional and expressive state preparation within that single qubit. With the increase of circuit layers, the parameter space also grows, which in other models flattens the gradient space, impeding parameter optimization. However, because the lack of entanglements limits the accumulation of nonlocal correlations between these parameters, the flattening of the model geometry is offset by the increase in the model expressivity; hence, the parameter space remains navigable for the optimizer (as evidenced from training R^2=0.7768 and testing R^2=0.5166 at 9 layers and the gradual improvement in training R^2=0.9977 at 27 layers and testing R^2=0.9192 at 21 layers).

By increasing the number of layers, the *serial* models reached the training and testing performance of the *cnn* models (classical neural networks).

7.1.2 Parallel models

In experiments with curve-fitting *parallel* models (see Figure 5 and Table 4), the circuit architectures were varied by changing the number of qubits (q), layers of trainable pre-encoding blocks (bl) and layers of trainable post-encoding blocks (al).

The experimental results were structured into four sections that corresponded to four groups of experiments.

G1. Experiments varying a number of qubits, while maintaining a fixed size number of pre-encoding layers (3) and post-encoding layers (3) (that number was picked at the middle of the intended testing range). As single-qubit *parallel* models are indistinguishable from *serial* models, so they were not considered, and thus the constructed models featured 2, 3, 4 and 5 qubits. Circuits with less than five qubits generated negative R^2 scores (worse than random). At five qubits, all the *parallel* model scores became meaningful, although indicating their low test performance (R^2=0.6957). It is evident from this experiment that the model expressivity grows with the number of qubits and the model performance improves in training and testing. For the remaining experiments, the number of qubits was fixed at five (5).

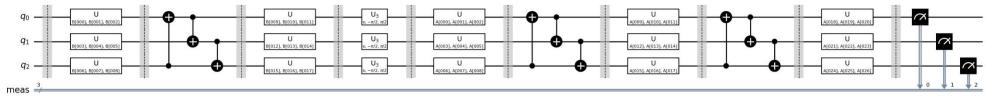

Fig. 5 Sample parallel model (Arch specs: q3 bl1 al2)

Table 4 Performance of a "parallel" (2-sins) model type

		Training		Testing		Arch	
Gr#	Qubits	Params	R^2	MSE	R^2	MSE	Specs
G1	2	48	0.0481	0.0456	-0.1354	0.0354	q2 bl3 al3
	3	72	0.0569	0.0452	-0.1532	0.0359	q3 bl3 al3
	4	96	-0.0177	0.0488	-0.2536	0.0391	q4 bl3 al3
	5	120	0.8460	0.0074	0.6957	0.0095	q5 bl3 al3
G2	5	60	0.7991	0.0096	0.7100	0.0090	q5 bl1 al1
	5	120	0.8460	0.0074	0.6957	0.0095	q5 bl3 al3
	5	180	0.7473	0.0121	0.6870	0.0098	q5 bl5 al5
	5	240	0.8397	0.0077	0.6697	0.0103	q5 bl7 al7
G3	5	60	0.7991	0.0096	0.7100	0.0090	q5 bl1 al1
	5	90	0.8145	0.0089	0.7137	0.0089	q5 bl1 al3
	5	120	0.7988	0.0096	0.7035	0.0092	q5 bl1 al5
	5	150	0.8149	0.0089	0.7052	0.0092	q5 bl1 al7
G4	5	90	0.8145	0.0089	0.7137	0.0089	q5 bl1 al3
	5	120	0.8460	0.0074	0.6957	0.0095	q5 bl3 al3
	5	150	0.8409	0.0076	0.7099	0.0090	q5 bl5 al3
	5	180	0.7526	0.0119	0.6887	0.0097	q5 bl7 al3

G2. Experiments varying an equal number of pre- and post-encoding layers, while maintaining a fixed number of qubits (5). Balanced *parallel* models favored small circuits, in terms of the number of parameters and layers, that is, they had better training performance (at 3 pre- and post-encoding layers, R^2=0.8460) and their generalizability according to test data was higher. The best performing *parallel* model had one (1) pre-encoding layer and one (1) post-encoding layer (R^2=0.7100), resulting in only 60 trainable parameters. This means that with more layers, the model gained more parameters and increased its entanglement. Although training performance was erratic, test performance was continually decreasing. In the absence of repeated sequential data reuploading, increasing the parametrization and entanglement of the model in the pre- and post-encoding layers was clearly to the detriment of the *parallel* model expressivity and trainability.

G3. Experiments varying the number of layers post-encoding, while maintaining a fixed number of qubits (5) and layers pre-encoding (1). From the previous section it was clear that single pre and post-encoding layers models performed best (so, pre-encoding layers were fixed at 1). Performance results in training and testing were erratic. A moderate gain in model expressivity was observed as a consequence of the increased parameterization of post-encoding layers (at 7 post-encoding layers, R^2=0.8149). However, the model quickly overtrained, so its best test performance was found early in the evolution of the model (at 3 post-encoding layers, R^2=0.7137).

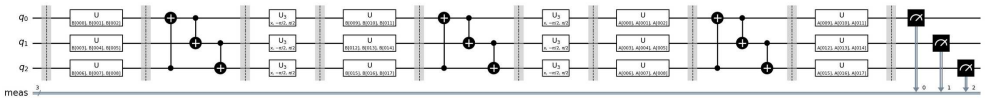

Fig. 6 Sample xparallel model (Arch specs: q3 bl2 al1)

Table 5 Performance of a "xparallel" (2-sins) model type

		Training		Testing		Arch
Qubits	Params	R^2	MSE	R^2	MSE	Specs
1	69	0.9950	0.0002	0.9448	0.0017	q1 bl21 al1
3	81	0.9980	0.0001	0.9823	0.0006	q3 bl7 al1
5	75	0.9614	0.0018	0.9043	0.0030	q5 bl3 al1
7	84	0.8541	0.0070	0.7236	0.0086	q7 bl2 al1
9	81	0.6796	0.0154	0.5897	0.0128	q9 bl1 al1

G4. Experiments varying the number of pre-encoding layers, while maintaining a fixed number of qubits (5) and post-encoding layers (3). The aim was to investigate whether or not more complex circuit initialization could have a significant impact on the generalizability of the model. However, the experiments revealed that for these data and the model architecture, a simple initialization was the most beneficial for the test results (at 1 pre-encoding layer and 3 post-encoding layers, test R^2=0.7137). Increasing parametrization and entanglement of the pre-encoding state preparation harms model trainability.

The *parallel* model results, in contrast to those obtained from the *serial* model, it is clear that both models rely on the preparation of a highly dimensional and expressive state due to repeated data reuploading. However, while the *serial* models increase the space dimensionality with every added layer, the *parallel* models operate in a fixed space defined by the number of qubits. Furthermore, with the addition of layers, together with the model parameters and entanglement, the *parallel* models suffer from the flattening of the gradient space and nonlocal interactions, without the compensation of the increased expressivity.

7.1.3 Xparallel models

Xparallel curve-fitting models were designed to extend the *parallel* models by adding sequential data reuploading. Experiments with these multi-qubit models (see Figure 6 and Table 5) focused on varying the number of qubits (q) and layers consisting of pairs: parallel reuploading blocks and parametrized training blocks (bl). The sequence of entangling and reuploading layers ended with optional parametrized entangling blocks (al, which were limited to 1).

As evidenced from the experiments carried out, the *xparallel* model favored circuits that are narrow and deep (at 3 qubits and 7 layers, training R^2=0.9980 and testing R^2=0.9823), even though they all had a similar number of trainable parameters (69-84). Note, however, that in this model, the circuit has more data reuploading blocks when the number of qubits decreases and fewer when the circuit narrows down but

increases its depth. With more data reuploading, the circuit created a highly expressive feature map, which compensated for the loss of Hilbert space dimensionality and expressivity within the ansatz.

A single qubit *xparallel* model was essentially *serial*, however, due to a different architecture (with more trainable parameters), it achieved a similar but different performance (training R^2=0.9950, testing R^2=0.9448).

7.2 Experiments with forecasting models

7.2.1 xqnn models

Xqnn models provide a small extension to the traditional QNNs to allow the addition of auxiliary qubits to allow the widening of circuits for trainable blocks to gain rotational parameters and entanglements (see Figure 7 and Table 5). The model circuits included an encoding multilayered ZZ feature map (fm) embedding windows of 5 input values (in), followed by a post-encoding layer of several trainable blocks (anz), with optional auxiliary qubits (to a total of q qubits). All qubits were measured. The experiments with *xqnn* models aimed to investigate their capacity to overparameterize, which was necessarily accompanied by an increase in model entanglement.

Three groups of experiments were conducted, each reported in a separate section of Table 6.

G1. Experiments that vary the number of *xqnn* model parameters by adding auxiliary qubits to its circuit (thereby increasing the width of the circuit). The best training performance was with two auxiliary qubits (at 7 qubits in total, training R^2=0.9764 and testing R^2=0.8474). By adding more auxiliary qubits and simultaneously more trainable parameters, the generalizability of the model continued to improve (at 9 qubits in total, testing R^2=0.8572). However, as the quantum simulator training of large *xqnn* models instances became extremely slow (e.g. 9 qubits and 5 trainable layers, with 162 parameters each), further increase of auxiliary qubits had to be abandoned. On reflection, with the addition of extra qubits, and hence model parameters and increased entanglement, the gradient space flattens, making the model optimization more difficult, which leads to the decrease in training performance. At the same time, with the addition of qubits, the model's Hilbert space expands exponentially offering richer feature space, so the model gains in its expressivity faster than its diminishing learning capacity, which explains a marginal improvement in test scores.

G2. Experiments that vary the number of *xqnn* models parameters by increasing the number of post-encoding trainable blocks (thus increasing the depth of the circuit). This entailed setting the number of trainable blocks (from 1 to 6), while fixing the total number of qubits (to $7 = 5$ for data encoding and 2 auxiliary), and keeping the encoding layer small (feature map with 1 ZZ layer). The experiments revealed that models of increasing depth and the consequent flattening of gradient space due to its dimensionality and high entanglement eventually started losing their learning capacity (from peak training R^2=0.9858 at 4 layers, down

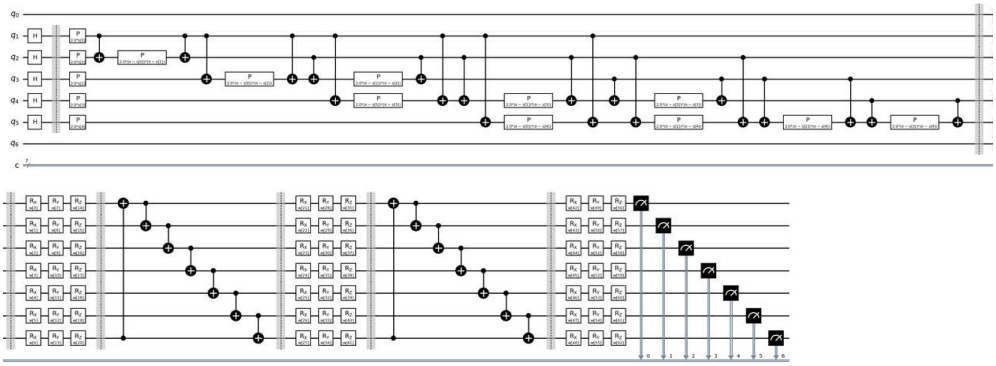

Fig. 7 Sample xqnn model (Arch specs: q7 in5 fm1 anz2)

Table 6 Performance of a "xqnn" (2-sins-sw) model type

			Training		Testing		Arch
Gr#	Qubits	Params	R^2	MSE	R^2	MSE	Specs
G1	5	90	0.9596	0.0016	0.6334	0.0128	q5 in5 fm1 anz5
	6	108	0.9276	0.0029	0.6833	0.0111	q6 in5 fm1 anz5
	7	126	0.9764	0.0009	0.8474	0.0053	q7 in5 fm1 anz5
	8	144	0.9399	0.0024	0.8122	0.0066	q8 in5 fm1 anz5
	9	162	0.9376	0.0025	0.8572	0.0050	q9 in5 fm1 anz5
G2	7	42	0.8043	0.0077	0.8443	0.0054	q7 in5 fm1 anz1
	7	63	0.9557	0.0017	0.8699	0.0046	q7 in5 fm1 anz2
	7	84	0.9603	0.0016	0.8796	0.0042	q7 in5 fm1 anz3
	7	105	0.9858	0.0006	0.8478	0.0053	q7 in5 fm1 anz4
	7	126	0.9764	0.0009	0.8474	0.0053	q7 in5 fm1 anz5
	7	147	0.9311	0.0027	0.8329	0.0058	q7 in5 fm1 anz6
G3	7	84	0.9603	0.0016	0.8796	0.0042	q7 in5 fm1 anz3
	7	84	0.7840	0.0085	0.5846	0.0145	q7 in5 fm2 anz3
	7	84	0.8427	0.0062	0.2726	0.0254	q7 in5 fm3 anz3

to R^2=0.9311 at 6 layers). Without the concurrent gain in the models' expressivity, their generalizability decreased as well (from the peak testing R^2=0.8796 at 3 layers, down to R^2=0.8329 at 6 layers).

G3. Experiments that vary the number of data encoding layers (from 1 to 3). The primary aim of the ZZ feature map is to encode input into a high-dimensional state, which maps differences and correlations in the classical data into a form suitable for the variation layers to clearly distinguish between data points features. However, as the input data are a time series window, the distinction between its neighboring values is small. Hence, encoding similar values into a highly entangled multilayer state preparation is likely to result in homogenized and randomized quantum states, making further state transformations meaningless. The effect is clearly visible in a sharp decrease in the model's ability to generalize from training data, in response to increased depth of the ZZ feature map (from testing R^2=0.8796 at 1 ZZ layer, down to R^2=0.2726 at 3 ZZ layers).

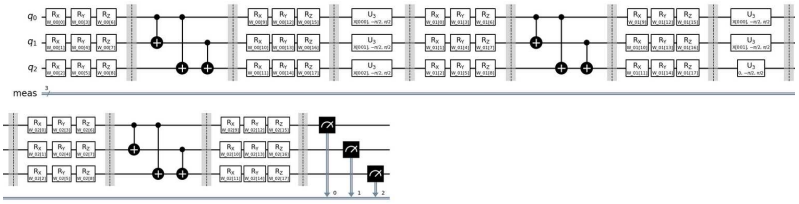

Fig. 8 Sample overloading model (Arch specs: q3 in5 xl1 il2)

Table 7 Performance of an "ovload" (2-sins-sw) model type

		Training		Testing		Arch
Qubits	Params	R^2	MSE	R^2	MSE	Specs
1	92	0.2316	0.0303	0.1560	0.0295	q1 xl3 il5
2	81	0.7848	0.0085	0.7697	0.0081	q2 xl2 il3
3	88	0.8719	0.0050	0.8504	0.0052	q3 xl2 il2
4	71	0.8839	0.0046	0.8668	0.0047	q4 xl1 il2
4	119	0.8866	0.0045	0.8723	0.0045	q4 xl2 il2
4	167	0.8890	0.0044	0.8735	0.0044	q4 xl3 il2

7.2.2 ovload models

Ovload models adopt the concept of multi-qubit data reuploading technique from the curve-fitting *xparallel* models. However, they tailor it to represent sliding TS windows, rather than individual TS values, to reduce the demand for qubit resources (see Figure 8 and Table 7). This is accomplished by splitting longer windows into a number of smaller fragments (il) that fit the available qubits (q). The *ovload* layers (xl) are then formed by encoding all window fragments and interleaving them with trainable blocks. As a result, the circuit's qubits are overloading the representational function for the windows sub-sequences, which relies on the continuity of reuploaded consecutive data points in a TS window (to replace their identity in the *xparallel* models). To deal with the threat of homogenization of TS features due to highly entangled state preparation (the issue noted in *xqnn* model encoding), the layers are being repeated to reupload the entire window's data, and hence increase the model expressivity.

Table 7 reports the experimental results of varying the number of *ovload* model qubits (from 1 to 4 ¡ 5, which is the window length) and re-uploading TS windows by repeating the model layers (from 1 to 3). When TS windows are encoded into a single qubit, the model mimics the behavior of a curve fitting *serial* model of 15 layers (xl=3, il=5). However, the discrepancy in the values of the represented windows prevents the model from forming a coherent quantum state, resulting in poor training performance (R^2=0.2316). As the qubit numbers approach those of the QNN representation (4 qubits to represent windows of 5 values), the model reached its test performance peak (testing R^2=8735), which was similar to that of *xqnn*. At the same time, the model doubled the number of parameters, thus flattening the gradient space, resulting in poor model optimization and inadequate learning capacity to reach its full potential (as evidenced by the suboptimal training R^2=0.8890). Furthermore, the objective of reducing the qubit resource was not met.

Table 8 Performance of a classical "cnn" (2-sins-sw) neural network

		Training		Testing		Arch
Qubits	Params	R^2	MSE	R^2	MSE	Specs
0	131	0.9987	0.0000	0.9763	0.0008	hl008 005 003
0	426	1.0000	0.0000	0.9772	0.0008	hl010 015 010
0	6181	1.0000	0.0000	0.9797	0.0007	hl050 080 020
0	20800	1.0000	0.0000	1.0000	0.0000	hl150 100 050

When the model halved the number of required qubits (to 3), the number of model parameters also halved, and the performance scores were close to the optimum (testing $R^2 = 0.8719$ and testing R^2=0.8504). However, this level of model performance was far from that of the *xqnn* model (best training R^2=0.9858 and testing R^2=0.8796).

7.2.3 cnn models

Experiments with classical *cnn* neural networks were included to provide a reference to compare with the performance of the remaining quantum models. All *cnn* models had three hidden layers with varying numbers of parameters. Four models were included, the "tiny" model (131 parameters), "small" (426 parameters), "medium" (6181 parameters) and "large" (20800 parameters).

The largest model was significantly overparameterized for the small data set used in the experiments. Consequently, all of its instances achieved near-perfect performance scores. The remaining *cnn* models also had near-perfect training scores; however, their test performance was below that of the curve fitting *xparallel* model (which had R^2=0.9823 vs R^2=0.9763 for "tiny", R^2=0.9772 for "small", R^2=0.9797 for "medium").

The smallest *cnn* model, which was comparable to quantum models with respect to the number of parameters, exceeded the test performance of both forecasting models, i.e. *xqnn* and *ovload*. However, the performance of the model architecture is based on the performance of its instance that achieves the median MSE test score, which is *cnn* instance #1 (Figure 9 in log scale). However, as can be seen from the distribution of MSE scores in all 10 instances, while the MSE scores of the four best performing instances (highlighted) are consistently close, the worst MSE scores are two magnitudes higher than the best. This means that low-parameterized *cnn* are highly sensitive to their initialization and cannot be trusted in practice. This is in contrast with the forecasting *xqnn* quantum model (Figure 10 in log scale, with four best instances highlighted), which benefits from the high-dimensional representation of its features in the Hilbert space, thus improving the model expressivity, and as a result the MSE of all instances converged within a consistent small range. Therefore, the quantum *xqnn* model could be trusted more than a small classical *cnn* model of comparable size in parameter space.

The analysis reviewed five types of QTSA models within the curve-fitting and forecasting model groups. It evaluated their architectural variants and their performance

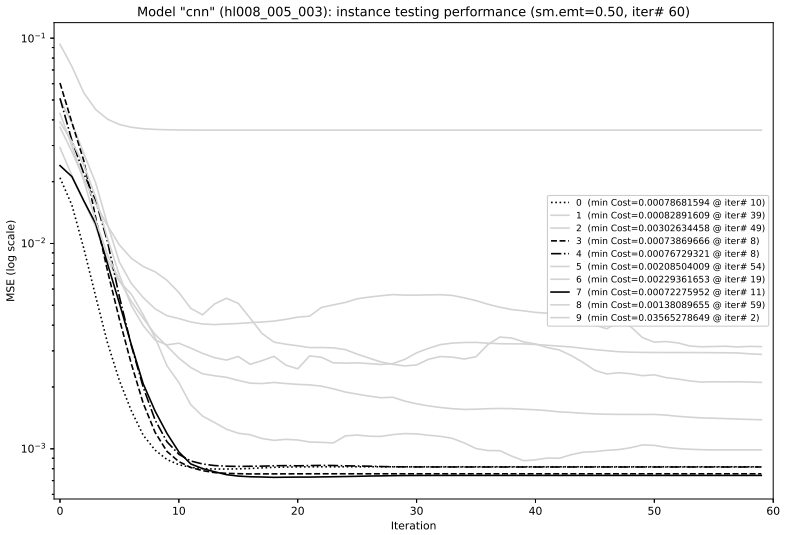

Fig. 9 Distribution of testing MSE for *cnn* model instances (hl008 005 003).

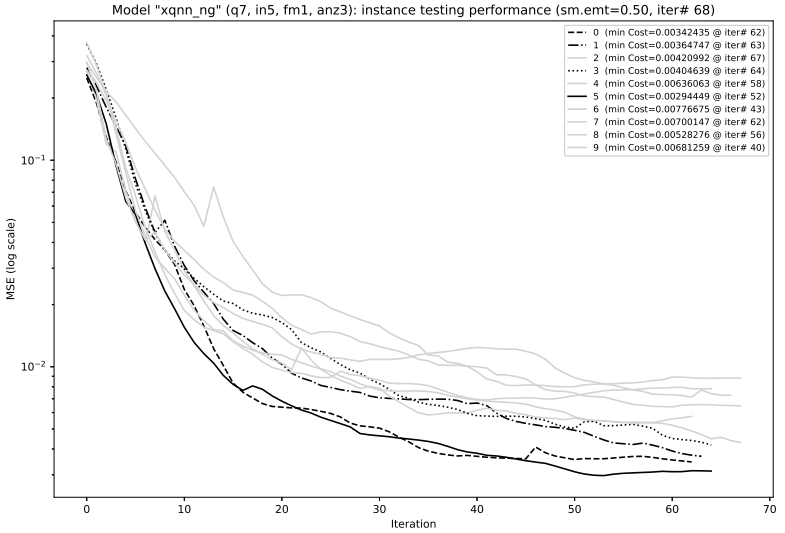

Fig. 10 Distribution of testing MSE for *xqnn* model instances (q7 in5 fm1 anz3).

with respect to the previously established design aspects. Most importantly, it established the connection of model design to its expressivity and trainability. The following section will provide a summary and synthesis of the insights derived from the reported research.

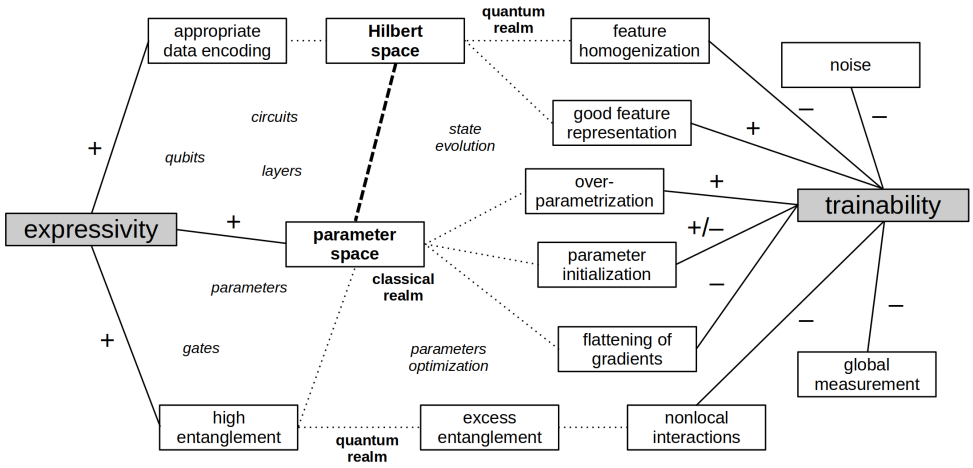

Fig. 11 Expressivity vs. trainability in QTSA models (positive and negative influences of design aspects)

8 Summary and conclusions

Due to the nature and volume of time series (TS) data, their analysis with quantum machine learning methods is challenging. This article discussed models for predictive quantum time series analysis (QTSA), their architectures, capabilities, and performance. In relation to predictive quantum time series analysis, the presented study has addressed two specific challenges in achieving the quality of QTSA models (see Section 1). The first was to recognize and understand the model **expressivity**, which describes its ability to effectively represent and process TS data. The second was the model **trainability**, that is, its capacity to learn and generalize for predictive accuracy, and efficiency in the process of model optimization.

Two groups of quantum models have been discussed, ***curve-fitting*** (*serial*, *parallel* and *xparallel*, see Section 7.1) trained to fit a function to sample data and ***forecasting*** (*xqnn* and *ovload*, see Section 7.2) able to predict future data from their preceding temporal context (sliding window) (see Sections 2 and 3). Within each group, models of different types have been designed, subsequently a set of custom-designed model architectures have been developed, and differently initialized model instances (10 per each model configuration) have been created, trained, tested, analyzed, evaluated and presented (see Section 7). In total, 500 individual models have been prepared and executed on the noise-free Qiskit state vector simulator. These included 260 quantum curve-fitting models, 200 quantum forecasting models, as well as 40 classical models developed for comparison and reference. All performance results of the developed models were collected, analyzed and summarized (see Section 6).

The experiments revealed many factors that influence the expressivity and trainability of the model (see Figure 11). These include all circuit elements (qubits, gates, and measurements), its structure (encoding and trainable blocks, and their layers), the function of its structural components (data encoding, state transformation, and measurement), and the processes involved in training and execution of circuits (such

as parameter optimization and state evolution). Some of these factors relate to the quantum realm (such as Hilbert space) and other factors to the classical realm (such as the parameter space used by the optimizer).

The first insight that is gained from the conducted experiments is that QTSA models need to adopt a data encoding strategy that is sensitive to the nature of the encoded TS data. As time series represent a continuum of changing values, the correlation between neighboring data points is high, and the distinction between them is small. To avoid homogenization of the encoded values in quantum space, two possible approaches are available: (1) avoiding data encoding complexity (as was the case in the *parallel* and *xqnn* models); and, (2) repeated reuploading of TS values to refresh their quantum representation in the Hilbert space (as was the case in all curve-fitting models and in the *ovload* model). Effective encoding of TS data improves model expressivity and indirectly supports its effectiveness of training.

As QTSA models tend to be large, both in depth and in width, their design needs to pay special attention to the deployment of entanglements for state preparation and within their ansatze. Entanglement of quantum circuits correlates their states and enriches the representation of encoded data, increasing the model expressivity. It also facilitates the evolution of these states towards the required measurement outcomes, influencing trainability. Despite the entanglement benefit to the model expressivity, when deployed in state preparation, it may harm the effectiveness of encoding temporal data (as mentioned above). With high level of entanglement, correlation of highly similar states resulting from encoding of highly similar TS values further scrambles the states that already lack separability and discernibility in Hilbert space. When it results from ansatz enlargement, e.g. for the purpose of gaining more trainable parameters, this may lead to excess entanglement, with inevitable nonlocal correlations between qubits and the parameters of their gates, thus causing optimization problems. Furthermore, as entanglement correlates the model states (via qubits), this in turn removes independence of parameters (attached to qubit gates). As these effects occur in separate domains of concern (quantum and classical), the effect is not explicitly visible to the optimizer. The following solutions to these problems are possible: (1) keeping data encoding layers narrow by adding auxiliary qubits for ansatz use only (as seen in the *xqnn* model); (2) reducing the impact of entangling gates by interleaving them with reuploaded data (as evidenced in the *xparallel* model); and (3) relying on a single-qubit solution and avoiding entangling gates altogether (*serial* model).

Altering the QTSA model function relies on a collection of its quantum circuit parameters, which are reflected in a navigable parameter space of the classical optimizer. A large number of parameters enhance the quantum model's ability to represent rich data features, thus boosting the model expressivity. At the same time, a large number of parameters increase the parameter space dimensionality, resulting in flattening gradients of the model geometry within the classical parameter space. In such a case, the optimization process faces a barren plateau in search for the optimum in the cost landscape, which is spanning the parameter space, and which severely impacts the model trainability. However, model training may also be impeded by local minima, preventing the optimizer from accessing a global solution. In those cases, adding more parameters beyond the normal training requirements may smooth the cost landscape

and, as a result, improve its navigability and hence trainability. There are many problems in this area, which have contradictory resolutions: (1) when facing insufficient model expressivity, increasing the ansatz width and depth will add more parameters to the circuit, which could be optionally combined with data reuploading (all curve-fitting models); (2) model overparameterization by the addition of auxiliary qubits can improve the model trainability by eliminating local minima in the model cost landscape (as available in the *xqnn* model); (3) there are many approaches to mitigating barren plateaus [15], however, reducing the circuit size in terms of its width and number of qubits, either by their reuse or overloading with data features, is also a viable option (as demonstrated in the *ovload* model).

A separate note needs to be added on the importance of model parameter initialization. As reported in this study, all experiments relied on multiple randomly initialized model instances (both quantum and classical). As observed (e.g., Figures 10 and 9), some instances resulted in vastly different levels of training and test performance. However, the significance of parameter initialization is a well-known aspect of quantum model development [15]. Importantly, the selection of the effective initialization approach could help avoid barren plateaus in training, but at the same time, may influence model design and its staged development, which we wanted to avoid. The study therefore adopted the "pessimistic" approach to the reporting of results, in which to deal with a wide range of instance performance scores, the scores of the median performing instance have been attributed to the model itself and the instance as its representative.

Other important QTSA design aspects have been left out of the scope of this study and are therefore not covered in the analysis and discussion. The most important are the choice of the circuit measurement strategy and the approach to noise mitigation in QTSA models (both included in Figure 11), which have been left for future work and extensions. There have also been many other experimental observations that could not be explained in a conclusive way and will require further exploration. For example, the erratic training and test performance of models in response to enlarging of their pre-encoding training layers in the absence of data reuploading (see *parallel* model); or the inability of the overloading quantum circuit (the *ovload* model) to rely on TS data redundancy to compensate for the reduction in its circuit width (and consequently the number of parameters).

In conclusion, this article explained several design aspects of quantum models for time series analysis. It indicated the link between their architectural characteristics and their expressiveness and trainability. It also demonstrated the experimental approach for identifying the type and configuration of the QTSA model that is most suitable for predictive analytical tasks. We hope that the outlined process will be of benefit to future practitioners and researchers of quantum time series analysis.

Declarations

- The authors have no competing interests to declare that are relevant to the content of this article.
- All data and programs which resulted from this project are in public domain.

References

[1] Masini, R. P., Medeiros, M. C. & Mendes, E. F. Machine learning advances for time series forecasting. *Journal of Economic Surveys* **37**, 76–111 (2023).

[2] Goodfellow, I., Bengio, Y. & Courville, A. *Deep Learning* (The MIT Press, 2016).

[3] Eldar, Y. & Oppenheim, A. Quantum signal processing. *IEEE Signal Processing Magazine* **19**, 12–32 (2002).

[4] Yarkoni, S., Kleshchonok, A., Dzerin, Y., Neukart, F. & Hilbert, M. Semi-supervised time series classification method for quantum computing. *Quantum Machine Intelligence* **3**, 12 (2021).

[5] Cuéllar, M. P., Pegalajar, M. C., Ruiz, L. G. B. & Cano, C. Rojas, I., Joya, G. & Catala, A. (eds) *Time Series Forecasting with Quantum Neural Networks.* (eds Rojas, I., Joya, G. & Catala, A.) *Advances in Computational Intelligence*, 666–677 (Springer Nature Switzerland, 2023).

[6] Liu, N. & Rebentrost, P. Quantum machine learning for quantum anomaly detection. *Physical Review A* **97**, 042315 (2018).

[7] Larocca, M., Ju, N., García-Martín, D., Coles, P. J. & Cerezo, M. Theory of overparametrization in quantum neural networks (2021). URL http://arxiv.org/abs/2109.11676. ArXiv:2109.11676 [quant-ph, stat].

[8] García-Martín, D., Larocca, M. & Cerezo, M. Effects of noise on the overparametrization of quantum neural networks (2023). URL http://arxiv.org/abs/2302.05059. ArXiv:2302.05059 [quant-ph, stat] version: 1.

[9] Pérez-Salinas, A., Cervera-Lierta, A., Gil-Fuster, E. & Latorre, J. I. Data re-uploading for a universal quantum classifier. *Quantum* **4**, 226 (2020).

[10] Schuld, M., Sweke, R. & Meyer, J. J. The effect of data encoding on the expressive power of variational quantum machine learning models. *Physical Review A* **103**, 032430 (2021).

[11] Wen, J., Huang, Z., Cai, D. & Qian, L. Enhancing the expressivity of quantum neural networks with residual connections. *Communications Physics* **7**, 220 (2024). URL https://www.nature.com/articles/s42005-024-01719-1. Publisher: Nature Publishing Group.

[12] Holmes, Z., Sharma, K., Cerezo, M. & Coles, P. J. Connecting Ansatz Expressibility to Gradient Magnitudes and Barren Plateaus. *PRX Quantum* **3**, 010313 (2022). URL https://link.aps.org/doi/10.1103/PRXQuantum.3.010313. Publisher: American Physical Society.

[13] Marrero, C. O., Kieferová, M. & Wiebe, N. Entanglement induced barren plateaus. *arXiv preprint arXiv:2010.15968* (2020).

[14] Cunningham, J. & Zhuang, J. Investigating and mitigating barren plateaus in variational quantum circuits: a survey. *Quantum Information Processing* **24** (2025). URL https://link.springer.com/10.1007/s11128-025-04665-1. Publisher: Springer Science and Business Media LLC.

[15] Cybulski, J. L. & Nguyen, T. Impact of barren plateaus countermeasures on the quantum neural network capacity to learn. *Quantum Information Processing* **22**, 442 (2023). URL https://doi.org/10.1007/s11128-023-04187-8.

[16] Bowles, J., Ahmed, S. & Schuld, M. Better than classical? The subtle art of benchmarking quantum machine learning models (2024). URL http://arxiv.org/abs/2403.07059. ArXiv:2403.07059 [quant-ph].

[17] DeCross, M., Chertkov, E., Kohagen, M. & Foss-Feig, M. Qubit-reuse compilation with mid-circuit measurement and reset (2022). URL http://arxiv.org/abs/2210.08039. ArXiv:2210.08039 [quant-ph].

[18] Weigold, M., Barzen, J., Leymann, F. & Salm, M. Encoding patterns for quantum algorithms. *IET Quantum Communication* **2**, 141–152 (2021).

[19] Schuld, M. & Killoran, N. Quantum machine learning in feature hilbert spaces. *Physical Review Letters* **122** (2019). URL http://dx.doi.org/10.1103/PhysRevLett.122.040504.

[20] Havlíček, V. *et al.* Supervised learning with quantum enhanced feature spaces. *Nature (on Arxiv)* **567**, 209–212 (2019). URL http://arxiv.org/abs/1804.11326.